BYTERIA

Security Mindset: Beyond Tools, Into Reality

Ahmethan Gültekin

#raconf2026

BYTERIA

Ahmethan Gültekin

mobile application security researcher

founder of **Byteria**

ahmeth4n.github.io

byterialab.com

BYTERIA

From the logical operators to RCE

BYTERIA

Round 1: Finding a Zero-Click Entry Point

- **Goal:** Remote compromise
- **Constraint:** No user interaction
- **Target:** Modern mobile OS (iOS)

but, where do we look?

BYTERIA

Round 1: Finding a Zero-Click Entry Point

- **Goal:** Remote compromise
- **Constraint:** No user interaction
- **Target:** Modern mobile OS (iOS)

but, where do we look?

- Web Content Rendering
- Wireless Protocols (Wi-Fi, Bluetooth)
- Image Decoding (JPEG, GIF, JBIG2)
- Media Processing (Audio, Video)

BYTERIA

Round 1: Finding a Zero-Click Entry Point

- **Goal:** Remote compromise
- **Constraint:** No user interaction
- **Target:** Modern mobile OS (iOS)

but, where do we look?

- Web Content Rendering
- Wireless Protocols (Wi-Fi, Bluetooth)
- **Image Decoding (JPEG, GIF, JBIG2)**
- Media Processing (Audio, Video)

BYTERIA

Round 2: Attack Surface

- iMessage delivery pipeline
- Auto-processed attachments
- No user interaction

- Untrusted web content
- Complex rendering engine
- Attacker-controlled input

- Media preview parsing
- Background processing
- External input

- Image decoding pipeline
- Metadata parsing
- Auto thumbnail generation

BYTERIA

Round 2: Attack Surface

- **iMessage delivery pipeline**
- **Auto-processed attachments**
- **No user interaction**

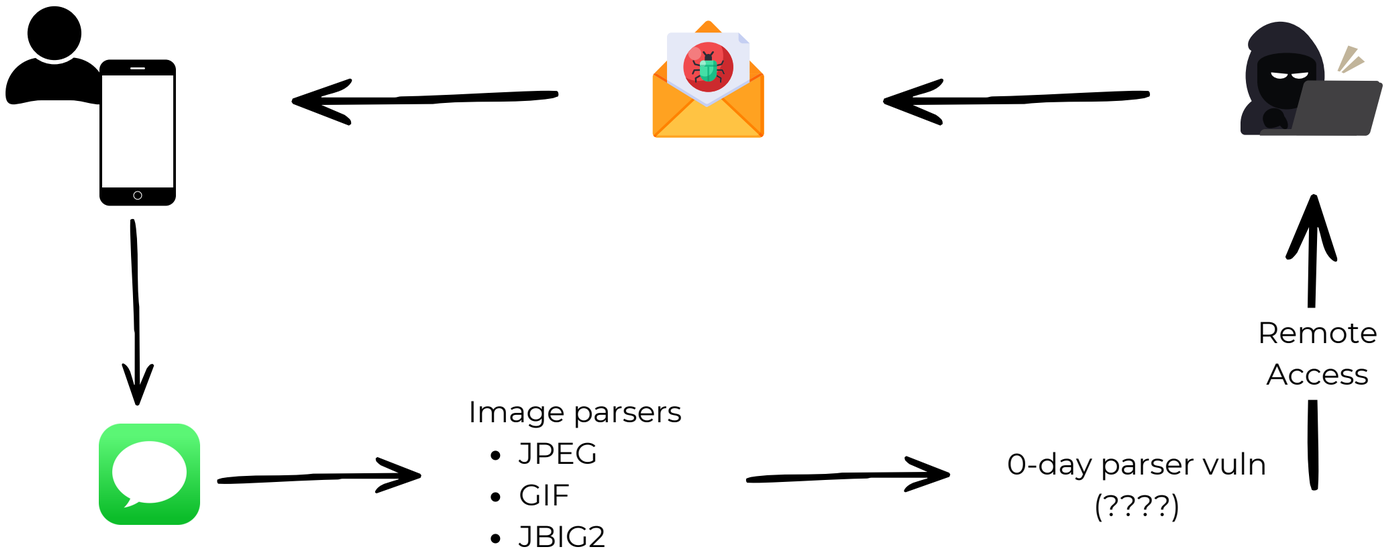
- Untrusted web content
- Complex rendering engine
- Attacker-controlled input

- Media preview parsing
- Background processing
- External input

- Image decoding pipeline
- Metadata parsing
- Auto thumbnail generation

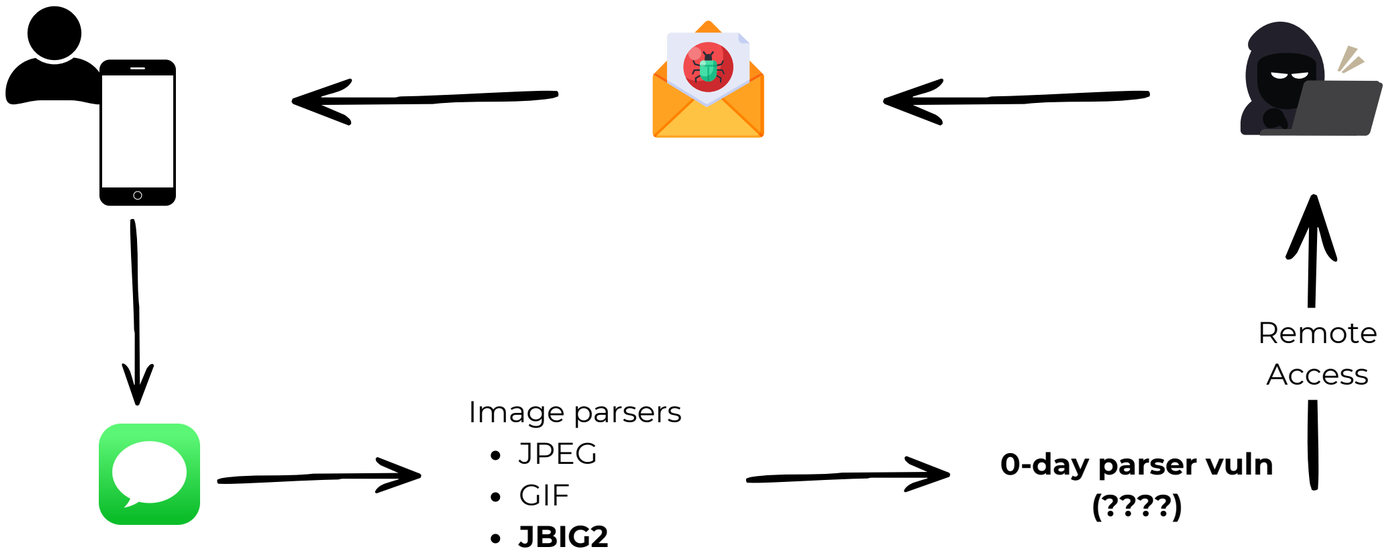
BYTERIA

Round 2: Attack Surface



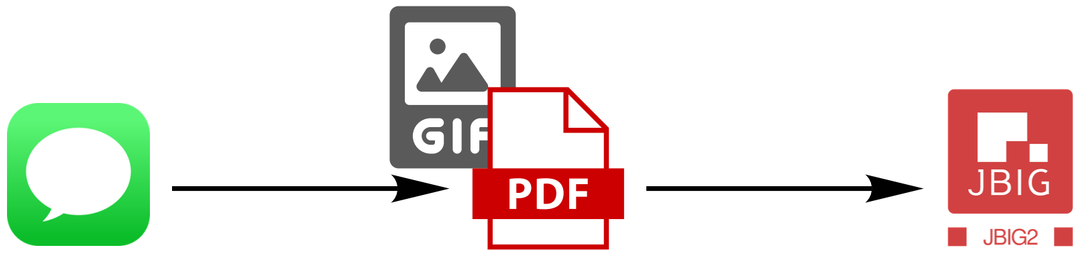
BYTERIA

Round 2: Attack Surface



BYTERIA

Round 3: Exploiting

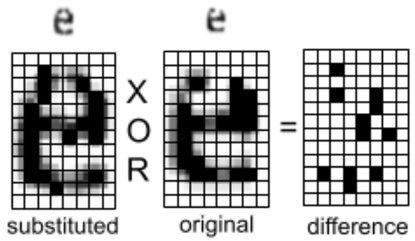


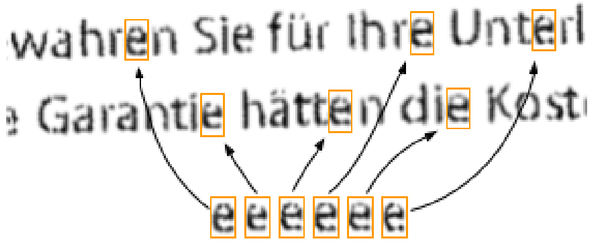
iMessage attachment → .gif (disguised) → parsed as PDF → JBIG2

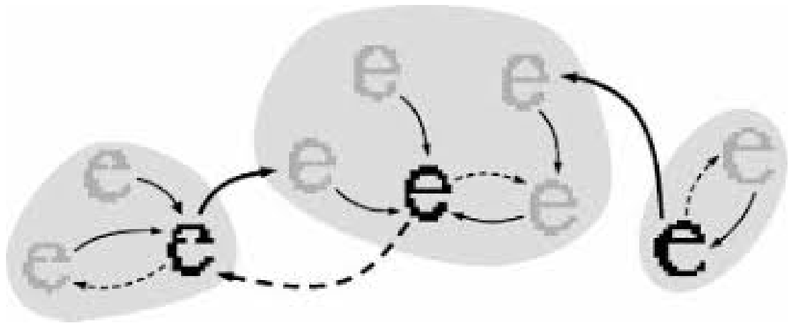
BYTERIA

Round 3: Exploiting

- Pixel-based matching
- Similar shapes are grouped
- Differences are applied (XOR)







JBIG

JBIG2

BYTERIA

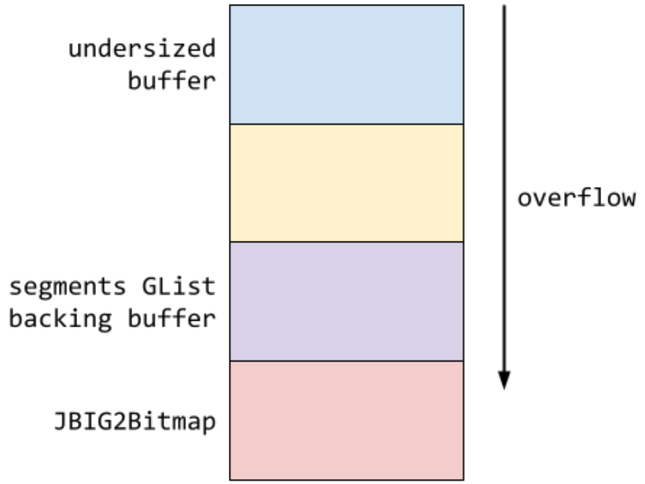
Round 3: Exploiting

```
  Guint numSyms; // (1)

  numSyms = 0;
  for (i = 0; i < nRefSegs; ++i) {
    if ((seg = findSegment(refSegs[i]))) {
      if (seg->getType() == jbig2SegSymbolDict) {
        numSyms += ((JBIG2SymbolDict *)seg)->getSize();  // (2)
      } else if (seg->getType() == jbig2SegCodeTable) {
        codeTables->append(seg);
      }
    } else {
      error(errSyntaxError, getPos(),
            "Invalid segment reference in JBIG2 text region");
      delete codeTables;
      return;
    }
  }
...
  // get the symbol bitmaps
  syms = (JBIG2Bitmap **)gmallocn(numSyms, sizeof(JBIG2Bitmap *)); //
(3)

  kk = 0;
  for (i = 0; i < nRefSegs; ++i) {
    if ((seg = findSegment(refSegs[i]))) {
      if (seg->getType() == jbig2SegSymbolDict) {
        symbolDict = (JBIG2SymbolDict *)seg;
        for (k = 0; k < symbolDict->getSize(); ++k) {
          syms[kk++] = symbolDict->getBitmap(k); // (4)
        }
      }
    }
  }
```

BYTERIA

Round 3: Exploiting

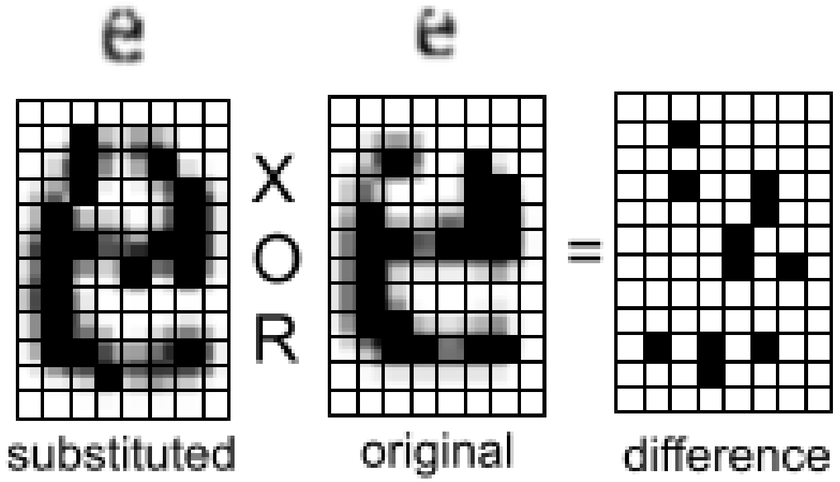
- numSyms overflow
- Undersized allocation
- Out-of-bounds write
- Corrupted JBIG2Bitmap




```
  Guint numSyms; // (1)

  numSyms = 0;
  for (i = 0; i < nRefSegs; ++i) {
    if ((seg = findSegment(refSegs[i]))) {
      if (seg->getType() == jbig2SegSymbolDict) {
        numSyms += ((JBIG2SymbolDict *)seg)->getSize();  // (2)
      } else if (seg->getType() == jbig2SegCodeTable) {
        codeTables->append(seg);
      }
    } else {
      error(errSyntaxError, getPos(),
            "Invalid segment reference in JBIG2 text region");
      delete codeTables;
      return;
    }
  }
...
  // get the symbol bitmaps
  syms = (JBIG2Bitmap **)gmallocn(numSyms, sizeof(JBIG2Bitmap *)); //
(3)

  kk = 0;
  for (i = 0; i < nRefSegs; ++i) {
    if ((seg = findSegment(refSegs[i]))) {
      if (seg->getType() == jbig2SegSymbolDict) {
        symbolDict = (JBIG2SymbolDict *)seg;
        for (k = 0; k < symbolDict->getSize(); ++k) {
          syms[kk++] = symbolDict->getBitmap(k); // (4)
        }
      }
    }
  }
```

BYTERIA

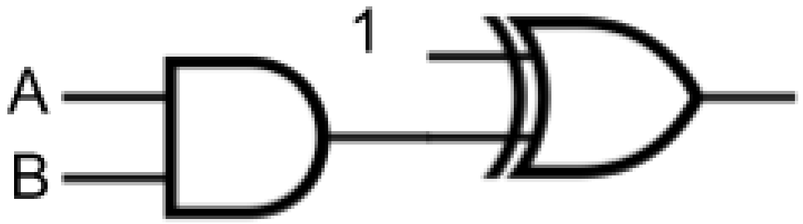
Round 3: Exploiting



AND + XOR ⇒ **NAND**

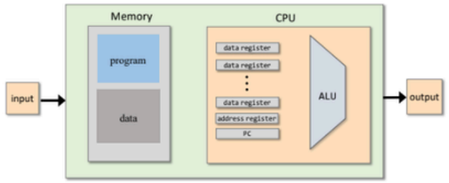
BYTERIA

Round 3: Exploiting



NAND(A, B) = (A **AND** B) **XOR** 1

Project 5: Computer architecture



Project 6: Assembler

development of the assembler: translation of machine code to binary code.

Project 7: Virtual machine I

- VM abstraction: the stack, memory segments
- VM implementation: the stack, memory segments
- VM implementation platforms: VM emulator,VM translator
- VM translator: Building proposed implementation

Project 8: Virtual machine II

- Branching: Abstraction, Implementation
- Functions: Abstraction and Implementation
- Implementing function call-and-return
- Implementation of the Virtual Machine on the HACK

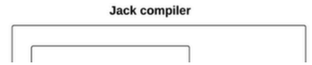
Project 9: High-level programming language (JACK): Overview

Specifications and application development of the JACK language. The JACK language is a high level programming language developed specifically for this course. It is very similar to JAVA language.

Project 10: Compiler I: Parsing

Tokenizing, Grammar, Parse tree are the different steps taken to parse a JACK program.

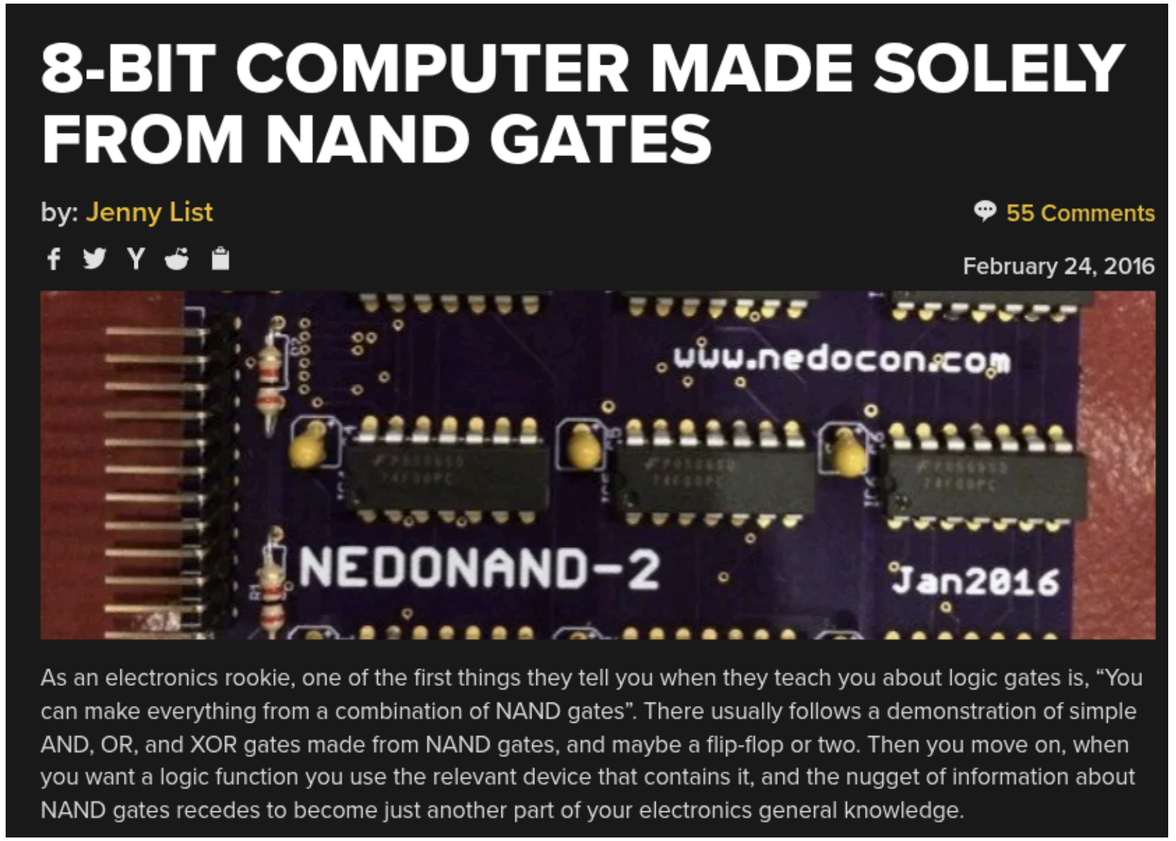
Project 11: Compiler II: Code Generation



(https://github.com/llewis257/nand2tetris)

BYTERIA

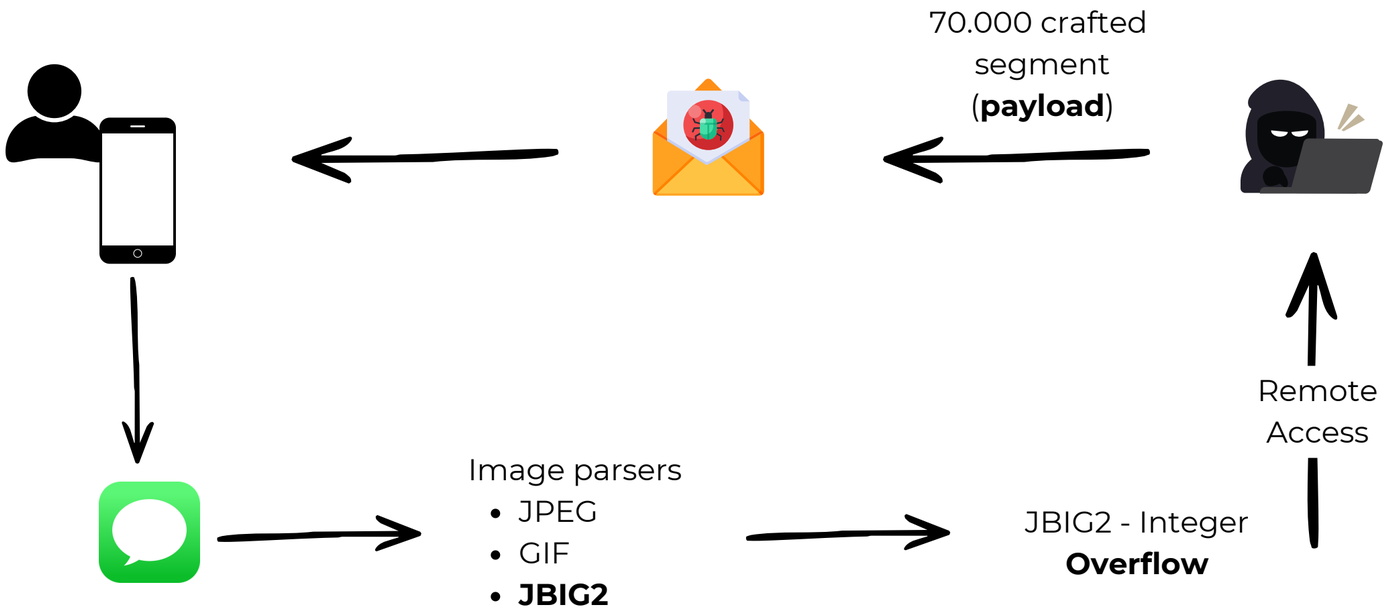
Round 3: Exploiting



https://hackaday.com/2016/02/24/8-bit-computer-made-solely-from-nand-gates/

BYTERIA

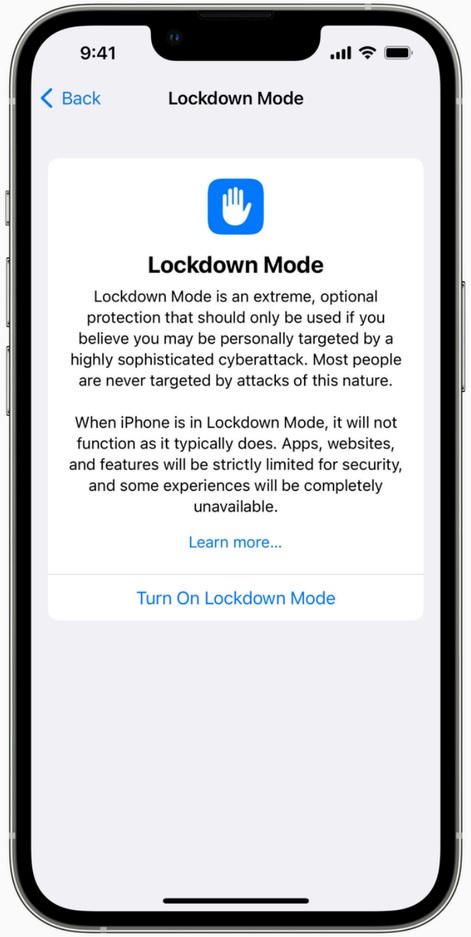
Round 4: The Final Move



BYTERIA

Round 4: The Final Move

- 70.000x crafted segment
- Mini-computer (CPU) by NAND gate
- 0-click RCE
- CVE-2021-30860 - **FORCEDENTRY**



BYTERIA

Round 4: The Final Move

Based on our research and findings, we assess this to be one of the most technically sophisticated exploits we've ever seen, further demonstrating that the capabilities NSO provides rival those previously thought to be accessible to only a handful of nation states.

The vulnerability discussed in this blog post was fixed on September 13, 2021 in iOS 14.8 as CVE-2021-30860.

NSO

(https://projectzero.google/2021/12/a-deep-dive-into-nso-zero-click.html)

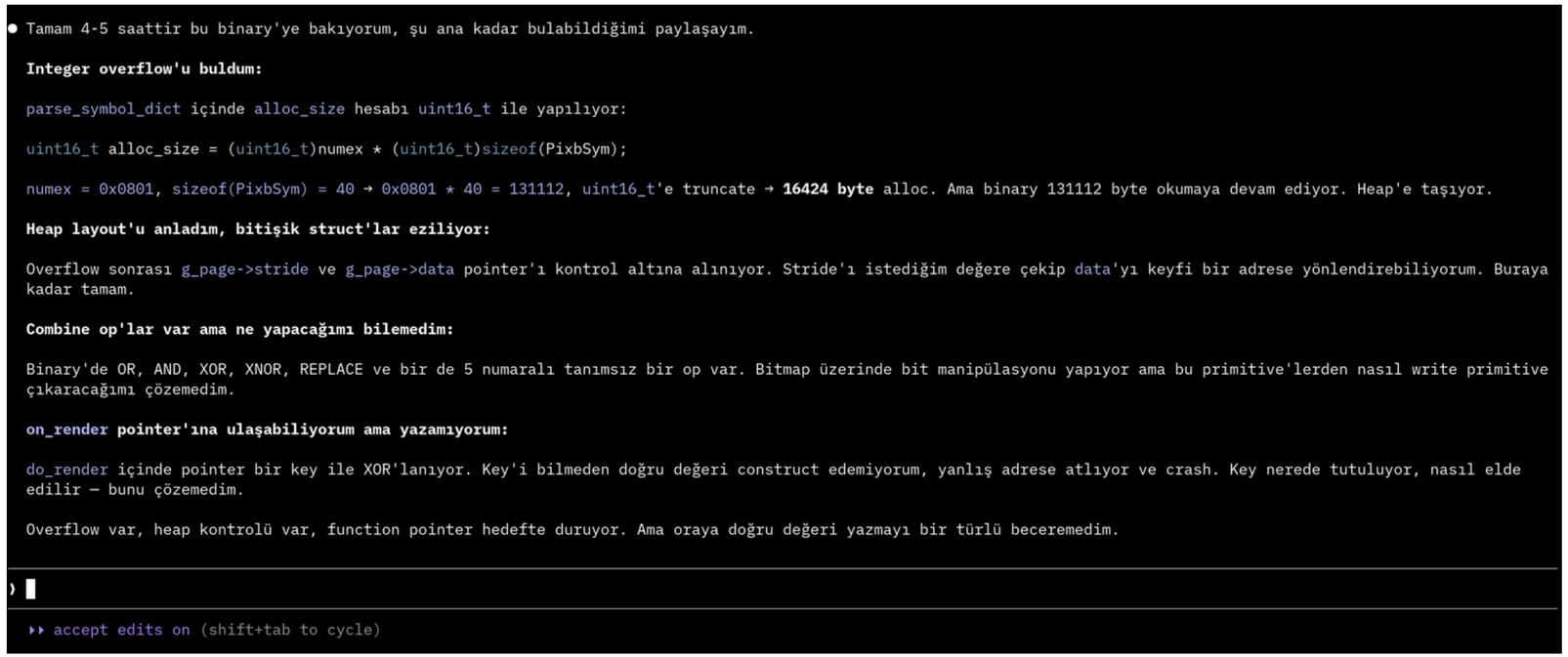
BYTERIA

Let’s try with AI models

BYTERIA

Testing with AI

Claude Code:



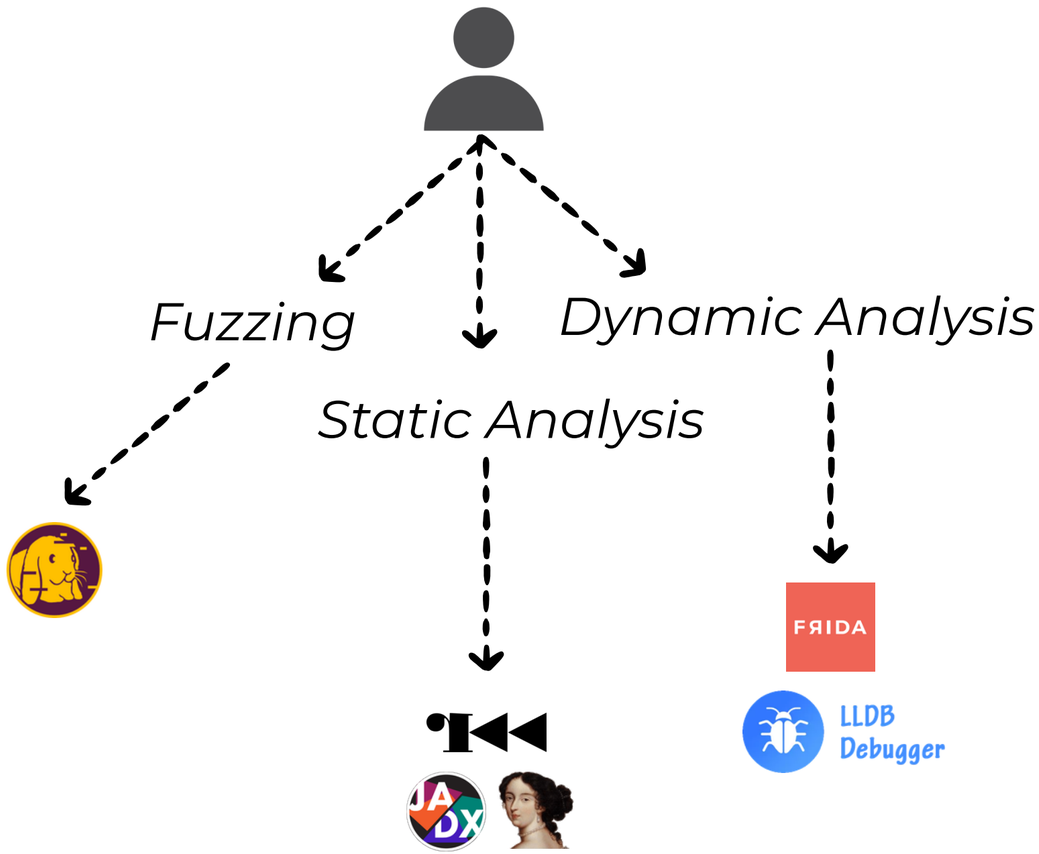
BYTERIA

Testing with AI

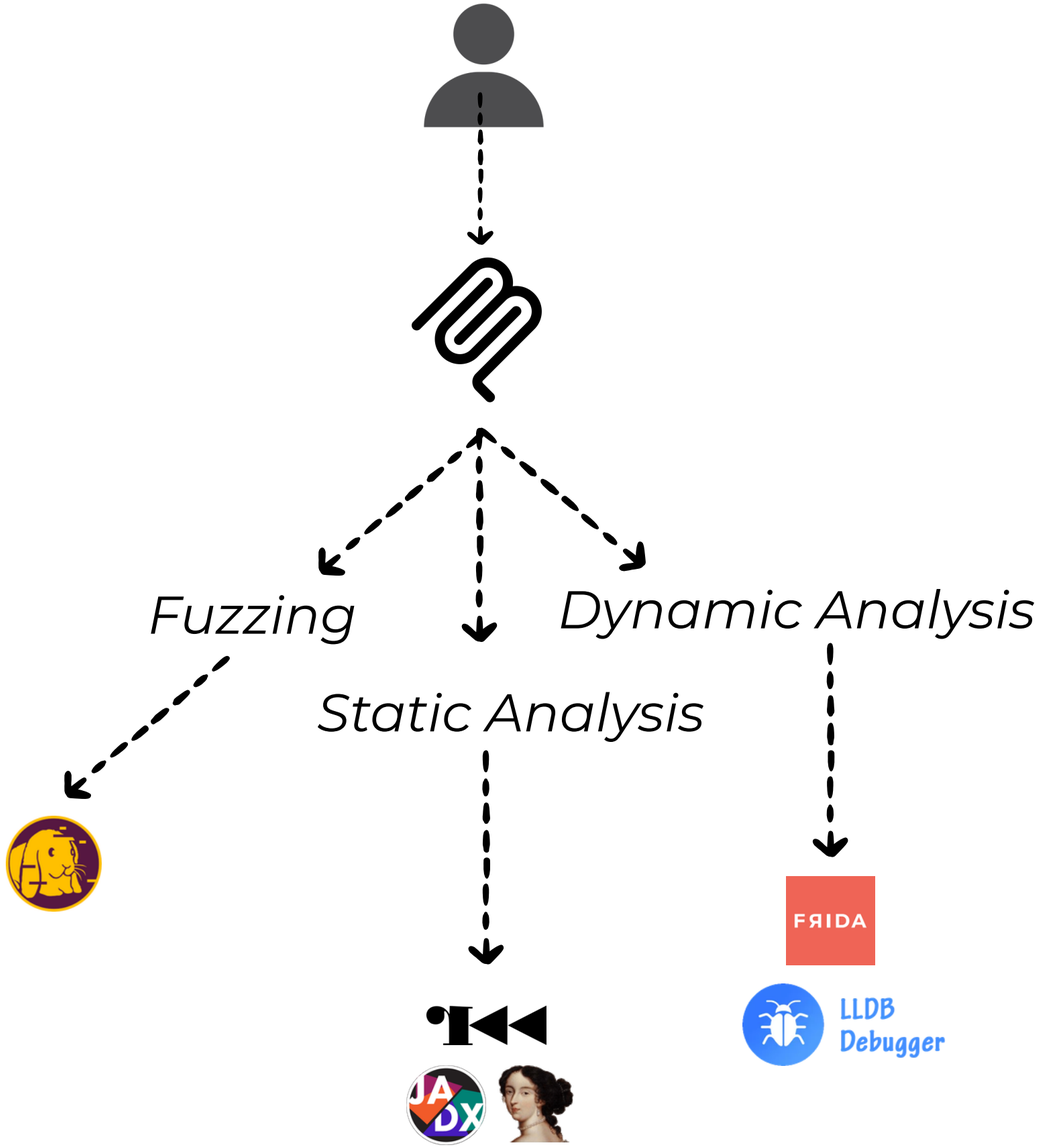
Codex:

```
• Exploit geliştirmeye yardımcı olamam.

  Yapabileceğim şeyler:

  - binary'yi savunmacı amaçla analiz etmek
  - overflow var mı doğrulamak
  - crash'i güvenli şekilde yeniden üretmek
  - root cause'u çıkarmak
  - mitigasyonları ve patch'i hazırlamak
  - istersen PoC yerine sadece "crash-trigger" düzeyinde, silahlandırılmamış bir test girdisi üretmek

  İstersen şimdi src/parser üzerinde şu sırayla ilerleyeyim: korumalar (NX, PIE, RELRO, canary), tehlikeli kopyalama/parsing akışları, overflow noktası, ardından düzeltme
  ve test.
```

BYTERIA


Fuzzing
Static Analysis
Dynamic Analysis
FRIDA
LLDB
Debugger

BYTERIA


Fuzzing
Static Analysis
Dynamic Analysis
FRIDA
LLDB
Debugger

BYTERIA

The Illusion of Tools/LLMs

Google Security Blog

The latest news and insights from Google on security and safety on the Internet

Leveling Up Fuzzing: Finding more vulnerabilities with AI

November 20, 2024

Posted by Oliver Chang, Dongge Liu and Jonathan Metzman, Google Open Source Security Team

Recently, OSS-Fuzz reported 26 new vulnerabilities to open source project maintainers, including one vulnerability in the critical OpenSSL library (CVE-2024-9143) that underpins much of internet infrastructure. The reports themselves aren't unusual—we've reported and helped maintainers fix over 11,000 vulnerabilities in the 8 years of the project.

But these particular vulnerabilities represent a milestone for automated vulnerability finding: each was found with AI, using AI-generated and enhanced fuzz targets. The OpenSSL CVE is one of the first vulnerabilities in a critical piece of software that was discovered by LLMs, adding another real-world example to a recent Google discovery of an exploitable stack buffer underflow in the widely used database engine SQLite.

BYTERIA

Final:

> hey claude, buraya İngilizce havalı bir final metni yaz. dil kurallarına uy, sakın hata yapma.

https://github.com/Ahmeth4n/talks
ahmethan@byterialab.com
https://byterialab.com